Projet - Utiliser Git et Github pour gérer son code source

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Création du repo local et distant **3**

A. Créer le repository sur GIthub 3

B. Cloner le repository local 4

C. Premiers commits 4

D. Premiers push 4

III. Gestion des branches **5**

A. Créer une branche 5

B. Pousser la branche sur GitHub 5

C. Faire des modifications sur la branche 5

D. Fusionner deux branches 6

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée (en minutes) : *60 minutes*

Environnement de travail : Git bash / Invit de commande

Pré requis :

Avoir Git et Visual Studio code installés sur la machine

Objectifs de la partie

- *Apprendre à initialiser un repository*
- *Apprendre à faire des commits*
- *Apprendre à travailler avec un repository distant*
- *Apprendre à gérer les branches*

Contexte

Dans ce projet, nous allons apprendre à utiliser Git. Le but sera de simuler la création d'un projet, et de voir comment utiliser Git dans toutes les étapes du développement. Pour faire tout cela, nous n'allons pas utiliser d'outils d'intégration de Git, mais nous allons apprendre à utiliser les commandes Git dans le terminal directement. Après ce projet, vous pourrez utiliser Git facilement lors de tous vos développements, ce qui vous permettra de travailler de façon plus sécurisée, et de pouvoir travailler à plusieurs.

II. Création du repo local et distant

A. Créer le repository sur GIthub

Repository distant

Lorsque vous utilisez Git pour gérer votre code, il est courant de stocker votre projet dans un repository distant. GitHub est une plateforme populaire pour l'hébergement de repositories Git. Lors de la création d'un repository sur GitHub, vous lui donnez un nom et choisissez les options de visibilité. Vous pouvez également choisir d'initialiser le repository avec un fichier README.md, qui sert souvent de documentation de base pour votre projet.

Méthode **Créer un repository sur Github**

Allez sur le site GitHub (GitHub[1]) et connectez-vous à votre compte. Cliquez sur le bouton « *Nouveau* » ou « *new* » pour créer un nouveau repository. Donnez un nom à votre repository, choisissez les options de visibilité et initialisez-le avec un fichier README.md. Vous pouvez entrer ces informations dans le Readme.md :

```
#Mon premier projet Git

Ce repository a été créé lors d'un projet étudiant pour tester Git et ses fonctionnalités.
```

Cliquez sur « *Créer le repository* » pour finaliser la création.

1 https://github.com

B. Cloner le repository local

Définition

Le clonage d'un repository signifie créer une copie locale du repository distant sur votre machine. Cela vous permet de travailler sur votre code localement et de le synchroniser avec le repository distant par la suite. Vous pouvez cloner un repository en utilisant la commande `'git clone <URL>'`, où <URL> est l'URL du repository distant.

Méthode

Ouvrez votre terminal ou votre interface en ligne de commande. Naviguez vers le répertoire où vous souhaitez cloner le repository. Sur la page GitHub du repository, cliquez sur le bouton « *Code* » et copiez l'URL du repository. Dans votre terminal, exécutez la commande suivante en remplaçant <URL> par l'URL que vous avez copiée. Cela créera une copie locale du repository sur votre machine.

```
1 git clone <URL>
```

C. Premiers commits

Définition

Un commit est un instantané de l'état actuel de votre projet. Il enregistre les modifications que vous avez apportées à vos fichiers. Lorsque vous effectuez un commit, vous donnez également une description significative des changements effectués. Les commits vous permettent de garder une trace de l'évolution de votre projet au fil du temps et de revenir en arrière si nécessaire. Pour effectuer un commit, vous utilisez les commandes `'git add <nom du fichier>'` pour ajouter les fichiers modifiés à l'index, puis `'git commit -m "Description du commit"'` pour créer le commit.

Méthode

Naviguez dans le répertoire cloné à l'aide de la commande cd <nom du répertoire>. Ajoutez dans ce dossier un fichier HTML. Utilisez la commande `git status` pour voir les fichiers modifiés.

```
1 git status
```

Utilisez la commande `'git add .'` pour ajouter tous les fichiers non indexés à l'index.

```
1 git add .
```

Utilisez la commande `'git commit -m "Description du commit"'` pour créer un commit avec les modifications. Assurez-vous d'inclure une description significative pour chaque commit.

```
1 git commit -m "Ajout du fichier index.html"
```

D. Premiers push

Définition

Une fois que vous avez effectué vos commits localement, vous pouvez pousser ces commits vers le repository distant sur GitHub en utilisant la commande `'git push'`. Cela envoie vos modifications sur le serveur distant, ce qui les rend accessibles à d'autres personnes travaillant sur le projet. Le push met à jour le repository distant avec vos derniers commits.

Méthode

Après avoir effectué vos commits, utilisez la commande git push pour pousser les commits vers le repository distant sur GitHub. Origin est le nom de votre remote, et main le nom de la branche que vous voulez pousser.

```
1 git push -u origin main
```


Tous droits réservés à STUDI - Reproduction interdite

Vous devrez peut-être vous authentifier en fournissant vos informations d'identification GitHub.

Une fois le push terminé, les modifications seront visibles sur le repository GitHub.

III. Gestion des branches

A. Créer une branche

Définition

Une branche est une version distincte de votre projet. Elle vous permet de travailler sur une fonctionnalité ou une tâche spécifique sans affecter la branche principale du projet. Lorsque vous créez une nouvelle branche, vous partez de la branche existante (généralement la branche principale) et vous travaillez indépendamment. Cela facilite la gestion des modifications parallèles et l'intégration ultérieure de ces modifications dans la branche principale. Pour créer une nouvelle branche, vous utilisez la commande `'git branch <nom de la nouvelle branche>'`.

Méthode

Assurez-vous d'être sur la branche principale (généralement appelée main ou master) en utilisant la commande `'git checkout <nom de la branche>'`.

Utilisez la commande `'git branch <nom de la nouvelle branche>'` pour créer une nouvelle branche à partir de la branche principale.

Vous pouvez choisir un nom significatif pour la nouvelle branche, en fonction de la fonctionnalité ou de la tâche sur laquelle vous travaillez.

B. Pousser la branche sur GitHub

Définition

Une fois que vous avez créé une nouvelle branche et effectué des modifications, vous pouvez pousser cette branche vers le repository distant sur GitHub. Cela vous permet de partager votre travail avec d'autres personnes et de collaborer plus facilement. Vous utilisez la commande `'git push -u origin <nom de la branche>'` pour pousser la branche vers le repository distant. L'option -u établit une relation de suivi entre la branche locale et la branche distante.

Méthode

Après avoir créé une nouvelle branche, utilisez la commande `git push -u origin <nom de la branche>` pour pousser la branche vers le repository distant sur GitHub.

Cela créera la branche sur le repository distant et la liera à votre branche locale.

C. Faire des modifications sur la branche

Définition

Une fois que vous êtes sur une branche spécifique, vous pouvez effectuer des modifications sur les fichiers de votre projet. Vous pouvez ajouter, supprimer ou modifier du code selon les besoins. Assurez-vous d'effectuer les commandes `'git add'`, `'git commit'` et `'git push'` pour enregistrer et pousser vos modifications vers le repository distant, tout comme dans la Partie 1 « *Création du repo local et distant* ».

Tous droits réservés à STUDI - Reproduction interdite

Méthode

Utilisez la commande `'git checkout <nom de la branche>'` pour vous déplacer sur la branche sur laquelle vous souhaitez travailler. Effectuez les modifications nécessaires dans les fichiers du repository. Ajoutez, committez et poussez les modifications de la même manière que dans la Partie 1 « *Création du repo local et distant* ».

D. Fusionner deux branches

Définition

Lorsque vous avez terminé de travailler sur une branche et que vous souhaitez intégrer vos modifications dans la branche principale (ou toute autre branche), vous pouvez effectuer une fusion. La fusion combine les modifications des deux branches et crée un nouveau commit pour représenter la fusion. Pour fusionner une branche avec une autre, vous utilisez la commande `'git merge <branche à fusionner>'`. Si des conflits surviennent pendant la fusion, vous devrez les résoudre manuellement en modifiant les fichiers concernés. Une fois la fusion terminée, vous effectuez un commit pour finaliser le processus.

Méthode

Pour fusionner une branche avec une autre (par exemple, fusionner une branche de fonctionnalité dans la branche principale), utilisez la commande `'git checkout <branche de destination>'` pour vous placer sur la branche de destination.

Ensuite, utilisez la commande `'git merge <branche à fusionner>'` pour fusionner la branche spécifiée avec la branche de destination.

Résolvez les éventuels conflits de fusion qui peuvent survenir.

Effectuez un commit pour finaliser la fusion en utilisant la commande `'git commit -m "Fusion de la branche X avec la branche Y"'`.

Pour aller plus loin :

Grâce à ce projet, nous avons pu voir comment fonctionne le processus de développement d'un site web en utilisant Git. Le développement avec Git ne se fait pas uniquement lors de la création du projet, mais tout au long du projet. Ces étapes que nous avons vues (add, commit, push, gestion des branches) sont des étapes qu'il faudra répéter cycliquement tout au long du développement.

Il y a différentes manières d'utiliser Git, et voici quelques idées pour continuer votre apprentissage de cet outil si pratique, et si important :

- **Mettre vos anciens projets Git :** vos précédents projets ne sont pas encore sur Git ? Profitez de ce moment pour tous les passer sur Git, vous êtes sûrs de les conserver en sécurité, et de pouvoir les continuer de façon propre.
- **Continuez ce projet :** ce projet est un projet web classique, vous pouvez décider de continuer ce projet, en ajoutant du contenu à cette page, en essayant de faire des commits le plus régulièrement possible.
- **Utiliser Git avec votre IDE :** vous pouvez essayer d'utiliser Git directement sur votre outil de développement. Par exemple, Visual Studio code embarque nativement une gestion de Git.

Tous droits réservés à STUDI - Reproduction interdite